

СХЕМЫ ЗАПРОСОВ (API & WebRTC) — V1.0 COMPLETE

Дата: 2026-06-24 | **Версия:** 1.0 (Production Ready)

Автор: Koda (AI) + NLP-Core-Team

Назначение: API Contract, WebSocket Events, WebRTC Signaling.

1. ОБЩИЕ ПРАВИЛА

1.1 Base URL

```
Production: https://api.balloo.su
Staging:    https://api-staging.balloo.su
Dev:        http://localhost:4000
```

1.2 Headers (Auth)

Все защищенные endpoints требуют заголовка:

```
Authorization: Bearer <JWT_TOKEN>
Content-Type:  application/json
```

1.3 Standard Response Format

Успех (200/201/204):

```
{
  "success": true,
  "data": { ... },
  "meta": {
    "page": 1,
    "limit": 20,
    "total": 100
  }
}
```

Ошибка (4xx/5xx):

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Неверный email",
    "details": [
      { "field": "email", "issue": "Invalid format" }
    ]
  }
}
```

2. AUTH & CODES (Вход и Коды)

2.1 Отправка кода (POST /api/auth/send-code)

Отправка SMS или Email с кодом подтверждения

Request Body:

```
{
  "identifier": "+79990000000", // Phone or Email
  "method": "sms" // 'sms' or 'email'
}
```

Response (200 OK):

```
{
  "success": true,
  "data": {
    "code_length": 6,
    "expires_in": 300 // seconds
  }
}
```

2.2 Верификация кода (POST /api/auth/verify-code)

Проверка кода и создание/авторизация сессии

Request Body:

```
{
  "identifier": "+79990000000",
  "code": "123456",
}
```

```
"device_info": {
  "name": "iPhone 13",
  "os": "iOS 17.0"
}
```

Response (200 OK):

```
{
  "success": true,
  "data": {
    "user": { ... },
    "token": "eyJhbGciOiJIUzI1NiIs... ",
    "refreshToken": "dGhpcyBpcyBhIHJlZnJlc2gg..."
  }
}
```

2.3 История входов (GET /api/me/login-history)

Просмотр логов авторизаций

Response (200 OK):

```
{
  "success": true,
  "data": [
    {
      "id": "log-uuid",
      "ip_address": "192.168.1.1",
      "user_agent": "Mozilla/5.0...",
      "location": { "city": "Moscow", "country": "RU" },
      "success": true,
      "created_at": "2026-06-24T10:00:00Z"
    }
  ]
}
```

3. CONTACTS (Синхронизация)

3.1 Синхронизация контактов (POST /api/contacts/sync)

Отправка хешей номеров для поиска пользователей

Request Body:

```
{
  "contacts_hashes": [
    "a1b2c3d4e5f6...", // SHA-256 of "+79990000001"
    "f6e5d4c3b2a1..." // SHA-256 of "+79990000002"
  ]
}
```

Response (200 OK):

```
{
  "success": true,
  "data": [
    {
      "phone_hash": "a1b2c3d4...",
      "user_id": "user-uuid-1",
      "display_name": "Иван Иванов",
      "avatar_url": "..."
    }
  ]
}
```

4. CHATS & MESSAGES (Обновленная логика)

4.1 Создание чата с паролем (POST /api/chats)

Request Body:

```
{
  "type": "PRIVATE",
  "password": "MySecretPass", // Если нужен пароль
  "participants": ["user-uuid-1"] // Для групп
}
```

4.2 Отправка сообщения с пересылкой (POST /api/chats/:id/messages)

Request Body (с пересылкой):

```
{
  "type": "TEXT",
  "content": "Привет!",
}
```

```
// Пересылка
"forward": {
  "from_user_id": "user-uuid-from",
  "message_id": "msg-uuid-orig"
},

// Ответ (Reply)
"reply_to_id": "msg-uuid-reply"
}
```

4.3 Загрузка вложений (POST /api/chats/:id/messages/attachments)

Request Body (multipart/form-data):

```
file: [Binary Data]
type: IMAGE | VIDEO | DOCUMENT
```

Response (200 OK):

```
{
  "success": true,
  "data": {
    "attachment_id": "att-uuid",
    "url": "https://storage.yandex.../file.jpg",
    "thumb_url": "https://storage.yandex.../thumb.jpg"
  }
}
```

5. INVITES (Приглашения)

5.1 Создание кода приглашения (POST /api/chats/:id/invites)

Request Body:

```
{
  "max_uses": 10,
  "expires_in_days": 7
}
```

Response (200 OK):

```
{
  "success": true,
  "data": {
    "invite_code": "A1B2C3D4",
    "invite_link": "https://balloo.su/join/A1B2C3D4"
  }
}
```

5.2 Присоединение по коду (POST /api/invites/:code/join)

6. WEBRTC SIGNALING (Calls)

Протокол: WebSocket (<ws://api.balloo.su/socket>)

6.1 Этапы звонка

1. **Invite:** `call_invite`
2. **Answer:** `call_answer`
3. **Signaling (SDP/ICE):** `call_signaling`
4. **End:** `call_hangup`

6.2 Структура сообщений (JSON)

`call_invite` (Отправитель)

```
{
  "type": "call_invite",
  "payload": {
    "chatId": "chat-uuid",
    "type": "VIDEO", // AUDIO, VIDEO
    "offer": { // SDP Offer (Base64 JSON)
      "sdp": "v=0\r\no=-... ",
      "type": "offer"
    }
  }
}
```

`call_answer` (Ответивший)

```
{
  "type": "call_answer",
  "payload": {
    "chatId": "chat-uuid",
    "answer": {
```

```

    "sdp": "v=0\r\no=-...",
    "type": "answer"
  }
}

```

call_signaling (Обмен ICE candidates)

```

{
  "type": "call_signaling",
  "payload": {
    "chatId": "chat-uuid",
    "candidate": {
      "candidate": "candidate:...",
      "sdpMid": "0",
      "sdpMLineIndex": 0
    }
  }
}

```

call_hangup (Завершение)

```

{
  "type": "call_hangup",
  "payload": {
    "chatId": "chat-uuid",
    "reason": "USER_ENDED" // USER_ENDED, NO_ANSWER, FAILED
  }
}

```

7. WEBSOCKET EVENTS (Real-time)

Соединение: `wss://api.balloo.su/socket`

7.1 Подключение

Client -> Server:

```

{
  "type": "connect",
  "payload": {
    "token": "JWT_TOKEN_HERE"
  }
}

```

```
}  
}
```

Server -> Client:

```
{  
  "type": "connected",  
  "payload": {  
    "userId": "user-uuid"  
  }  
}
```

7.2 События

Событие	Направление	Описание
<code>message_new</code>	Server -> Client	Новое сообщение (push + local)
<code>message_status</code>	Server -> Client	Статус (SENT/DELIVERED/READ)
<code>user_typing</code>	Bidirectional	<code>user_typing</code> (chatId), <code>user_stop_typing</code>
<code>call_invite</code>	Server -> Client	Входящий звонок
<code>emergency_alert</code>	Server -> Client	Экстренное оповещение
<code>story_new</code>	Server -> Client	Новый сторис от подписки

8. E2E KEY EXCHANGE

Проблема: Как клиент Б получает публичный ключ клиента А для шифрования?

Решение: Через API профиля.

1. **Клиент А** генерирует пару RSA/ECDH ключей локально.
2. **Клиент А** отправляет публичный ключ на сервер:

`POST /api/users/me/keys`

```
{  
  "publicKey": "-----BEGIN PUBLIC KEY-----\nMIIBIjAN..."  
}
```

3. **Клиент Б** запрашивает профиль А:
- `GET /api/users/A/profile`
4. **Ответ:** Содержит `publicKey`.

5. **Клиент Б** шифрует сообщение ключом А и отправляет.

⚠ **Важно:** Приватный ключ никогда не покидает устройство.

END OF REQUEST SCHEMES V1.0